

260415_2차 프로젝트_Unity 3기_우성혁



과제 제출 방법

1. 전체 복사 후 개인페이지에 붙여넣은 뒤 사용합니다.
2. 과제 작성 후 우측 상단에 점3개 아이콘을 클릭한 뒤 [내보내기] > [PDF] 로 노션 페이지를 변환합니다.
 - 내보내기 형식: PDF
 - 데이터베이스 포함: 현재 보기
 - 포함할 콘텐츠: 모든 콘텐츠
3. 다운받은 zip 파일 압축 해제 후, PDF 파일 명을 `yymmdd_2차 프로젝트_Unity 3기_000` 로 변경합니다.
4. 과제 제출란에 PDF 파일을 제출합니다. (zip 파일이 아닌, **PDF 파일**을 제출해주세요!)

**추후 포트폴리오, 프로젝트 문서에 이 과제들을 활용할 수 있으니 일일회고처럼 꼼꼼히 기록해주세요.*

Scrum

**금일 진행한 스크럼 내용을 간략하게 요약해주세요*

**스크럼을 진행하지 않았다면 공란으로 둡니다.*

Project

**프로젝트 작업에서 본인이 기여한 부분에 대해 작성합니다.*

**코드파일이 있다면 스크린샷이나 노션 코드를 사용하여 덧붙여주세요.*

(예: ERD 이미지, Figma 이미지, 코드 스크린샷, Unity 스크린샷 등)

1. 오늘 진행할 작업

FGT 플레이 가능하도록 개별 작업물 병합 후 이슈 해결

엔딩 조건 및 코드 최적화

2. 진행중인 작업

엔딩 조건 (기억 재구성 점수 + 기억 태그) 코드 개선

▼ 게임플레이 데이터 저장 코드

```
public void CreateSaveData(int slotNumber)
{
    curData.ID = (byte)slotNumber;
    curData.savedTime = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");
    string json = JsonUtility.ToJson(curData, true);
    File.WriteAllText(GetSavePath(slotNumber), json); //선택한 슬롯에 세이브 데이터를 저장
    File.WriteAllText(Path.Combine(Application.persistentDataPath, $"CurData.json"), json); //현재 데이터를 저장한 데이터로 갱신
    SaveUIManager.instance.CloseSavePopup();
    StartCoroutine(SaveUIManager.instance.SaveAlarm(slotNumber));
}

public static void CreateCurData(string path, SaveDataObj dataObj)
{
    dataObj.ID = 0;
    dataObj.savedTime = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");
    dataObj.memory_reconstruction_rate = new int[13];
    dataObj.memory_reconstruction_rate[0] = 5; //기억 재구성 기본 점수 5점으로 시작
    dataObj.CoreTag = CreateDefaultMemoryTags();
    dataObj.npcInformations = LoadDefaultNPCInfo();
    dataObj.isFirstEnterAtS3CP0 = false;
}
```

```

        dataObj.isFirstEnterAtEP3Lobby = false;
        dataObj.isFirstEnterAtEP3_1 = false;
        string json = JsonUtility.ToJson(dataObj, true);
        File.WriteAllText(path, json);
    }
    private static List<IsTagGet> CreateDefaultMemoryTags()
    {
        List<IsTagGet> tags = new List<IsTagGet>(DefaultMemoryTagNames.Length);
        foreach (string tagName in DefaultMemoryTagNames)
        {
            tags.Add(new IsTagGet
            {
                TagName = tagName,
                tagGet = false
            });
        }
        return tags;
    }
    public static List<NPCInfo> LoadDefaultNPCInfo()
    {
        TextAsset jsonFile = Resources.Load<TextAsset>("Data/npc_Info_default");
        if (jsonFile == null)
        {
            Debug.LogError("[SaveManager] npc_Info_default.json 파일을 찾을 수 없습니다. 경로를 확인하세요.");
            return new List<NPCInfo>();
        }
        NPCInfo[] npcArray = JsonHelper.FromJson<NPCInfo>(jsonFile.text);
        // talkCount, isUsed 초기화
        foreach (var npc in npcArray)
        {
            npc.talkCount = 0;
            if (npc.words != null)

```

```

        {
            foreach (var w in npc.words)
                w.isUsed = false;
        }
    }
    return new List<NPCInfo>(npcArray);
}

```

```

[
    {
        "npcId": "npc_ep1_luna",
        "Affinity": 50,
        "words": [
            { "word": "쌍둥이자리", "memoryRate": 10 }
        ]
    },
    {
        "npcId": "npc_ep2Painter",
        "Affinity": 50,
        "words": [
            { "word": "동료", "memoryRate": 1 },
            { "word": "작업실", "memoryRate": 1 },
            { "word": "색", "memoryRate": 1 },
            { "word": "?", "memoryRate": 1 },
            { "word": "!", "memoryRate": 1 }
        ]
    },
    {
        "npcId": "npc_ep3_musician",
        "Affinity": 50,
        "words": [
            { "word": "멜로디", "memoryRate": 2 },
            { "word": "리듬", "memoryRate": 2 },
            { "word": "하모니", "memoryRate": 1 }
        ]
    },
    {
        "npcId": "npc_ep4_core",

```

```

        "Affinity": 50,
        "words": [
            {"word": "기억", "memoryRate": 2},
            {"word": "동료", "memoryRate": 2},
            {"word": "하모니", "memoryRate": 2},
            {"word": "삶", "memoryRate": 2},
            {"word": "마지막 조각", "memoryRate": 2}
        ]
    }
]

```

▼ 기억 재구성 점수 분할 계산

```

        dataObj.memory_reconstruction_rate = new int
[13];
        dataObj.memory_reconstruction_rate[0] = 5;  //기
        억 재구성 기본 점수 5점으로 시작
        for (int i = 1; i < dataObj.memory_reconstructio
n_rate.Length; i++)  //나머지 기억 재구성 점수는 0점으로 시작
        {
            dataObj.memory_reconstruction_rate[i] = 0;
        }

```

```

        int totalPoint = 0;
        foreach(int i in SaveManager.instance.curData.me
memory_reconstruction_rate)  //저장된 기억 재구성 점수를 모두
        합산
        {
            totalPoint += i;
        }
        bool ReconstructionRateCond = totalPoint >= Endi
ngPoint();

```

▼ 최종 퍼즐 완료 시 계산 코드

```

public void Puzzle4Complete()  //퍼즐 완료 시 처리
{
    SyncCheck();
}

```

```

        puzzle4MemoryRate = puzzle4BaseMemoryRate - Math.
h.Clamp(retry_count, 0, puzzle4BaseMemoryRate); //다시하
기 횟수만큼 기억 퍼즐 재구성 점수 감점(최대 5점까지)
        SaveManager.instance.curData.memory_reconstructi
on_rate += puzzle4MemoryRate; //퍼즐 4 점수 획득
        SelfVoiceTag(egoSync >= 1f); //자아 통합도 100%
달성 여부에 따라 "self_voice" 태그를 획득
    }
    private void SelfVoiceTag(bool b)
    {
        var tag = SaveManager.instance.curData.CoreTag.F
irstOrDefault(t => t.TagName == "self_voice");
        if (tag != null) tag.tagGet = b;
        else
        {
            SaveManager.instance.curData.CoreTag.Add(new
IsTagGet
            {
                TagName = "self_voice",
                tagGet = b
            });
        }
        if (b == true)
        {
            Debug.Log($"마지막 기억 획득!");
            clearSound.Play();
            coreNPC.revealStage = MemoryRevealStage.Ful
l;
        }
    }
}

```

▼ NPC 대화 중 기억 재구성 점수 태그 감지 코드

```

        if (SaveManager.instance == null || SaveManager.
instance.curData == null || SaveManager.instance.curDat
a.npcInformations == null) return;
        NPCInfo npcInfo = SaveManager.instance.curData.n
pcInformations.Find(n => n.npcId == currentNPCData.npcI
d);

```

```

        if (npcInfo != null && npcInfo.words != null)
        {
            foreach (MemoryKeyword keyword in npcInfo.words)
            {
                if (string.IsNullOrEmpty(keyword.word))
                    continue;
                if (msg.IndexOf(keyword.word, StringComparison.OrdinalIgnoreCase) >= 0)
                {
                    Debug.Log($"[MemoryKeyword] 발견됨: {keyword.word}, isUsed={keyword.isUsed}");
                    if (!keyword.isUsed)
                    {
                        SaveManager.instance.curData.memory_reconstruction_rate += keyword.memoryRate;
                        keyword.isUsed = true;
                        SaveManager.instance.WriteCurJSON();

                        Debug.Log($"[MemoryKeyword] 증가! 현재 memory_reconstruction_rate = {SaveManager.instance.curData.memory_reconstruction_rate}");
                    }
                    else
                    {
                        Debug.Log($"[MemoryKeyword] 이미 사용됨. 증가 없음.");
                    }
                }
            }
        }
    }
}

```

▼ 현재 데이터(curData) 참조 코드 축약 예시 - Checkpoint_Plane

```

using System;
using System.Collections.Generic;
using UnityEngine;
public class Checkpoint_Plane : MonoBehaviour
{

```

```

    public int cpNum;
    public Transform spawnPos;
    public bool isCheck = false;
    private readonly string playerTag = "Player";
    public event Action S3FirstCheck;
    public static Dictionary<int, Dictionary<int, bool>>
cpProgress = new Dictionary<int, Dictionary<int, bool>>
();
    public SaveDataObj CurData;
    private void Start()
    {
        CurData = SaveManager.instance.curData;
    }
    private void OnTriggerEnter(Collider other)
    {
        if (other.gameObject.CompareTag(playerTag))
        {
            isCheck = true;
            SaveCheckpointProgress();
            if(cpNum == 0) S3FirstCheck?.Invoke();
        }
    }
    private void SaveCheckpointProgress()
    {
        if (SaveManager.instance == null) return;
        if (!cpProgress.ContainsKey(3)) cpProgress[3] =
new Dictionary<int, bool>();
        switch (cpNum)
        {
            case 0: cpProgress[3][0] = CurData.ep4_open;
break;
            case 1: cpProgress[3][1] = CurData.ep4_puzzl
e1Clear; break;
            case 2: cpProgress[3][2] = CurData.ep4_puzzl
e2Clear; break;
            case 3: cpProgress[3][3] = CurData.ep4_puzzl
e3Clear; break;
        }
    }

```



```

    }
    public static bool IsCheckpointCleared(int cpNum)
    {
        if (cpProgress.ContainsKey(3) && cpProgress[3].C
ontainsKey(cpNum))
        {
            return cpProgress[3][cpNum];
        }
        return false;
    }
}

```

3. 내일 개발 계획 및 목표

금요일 FGT 대비 main 브랜치에 통합 및 충돌 수정 작업

FGT 발표 자료 제작

Trouble Shooting

**금일 개발 과정 중, 문제를 해결한 경험이 있다면 기술해주세요.*

1. 문제 상황 파악
2. 문제 발생 조건 파악
3. 결과 검증, 해결

그 외 과제 제출 (선택기입)

Github / Blog

**별도로 회고를 정리하는 깃헙, 블로그 게시글 링크가 있다면 첨부해주세요.*

- Github:
- Blog:

- Velog:

기술탐구

**프로젝트 작업을 위해 추가로 학습한 게시글이 있다면 링크를 첨부해주세요*

- (예시) 부트스트랩 설치방법: <https://datamoney.tistory.com/149>
- 링크 1:
- 링크 2: